

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science & Engineering****ASSESSMENT TOOL 3 – OUTPUT PREDICTION**

Course Code	CSA0314	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 3: Output Prediction
Weightage	35% of CIA	Total Marks	100
CO1 Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)</i>		
Student Name	ARUTHRAN T	Reg. No.	192525416
Section/Batch	Slot C / 2025	Date	10-10-2026

Code of Conduct:

I ARUTHRAN T, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ARUTHRAN T

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Output Accuracy	30%	Predicts all program outputs correctly, including exact values and output sequence.	Predicts most outputs correctly with minor errors.	Predicts some outputs correctly but makes significant errors.	Fails to predict most program outputs correctly.	
Conceptual Understanding	25%	Demonstrates excellent understanding of ADT, arrays, recursion, data structures, and C programming concepts.	Demonstrates good understanding with minor conceptual errors.	Demonstrates partial understanding with several conceptual errors.	Demonstrates inadequate understanding of fundamental concepts.	
Logical Tracing and Execution Flow	20%	Traces loops, recursive calls, array updates, and function execution accurately and systematically.	Traces execution correctly with a few minor mistakes.	Shows incomplete tracing with errors in execution flow.	Unable to trace program execution logically.	
Time and Space Complexity Analysis	15%	Correctly identifies and justifies best-, average-, and worst-case complexity and auxiliary space requirements.	Identifies most complexities correctly with minor errors.	Identifies some complexities but provides incomplete or incorrect reasoning.	Unable to identify or explain algorithm complexity.	
C Syntax and Runtime Behavior	10%	Correctly interprets C expressions, variable updates, function calls, and undefined behavior.	Understands most C execution rules with minor mistakes.	Shows limited understanding of execution rules and expression evaluation.	Misinterprets fundamental C language rules and program behavior.	

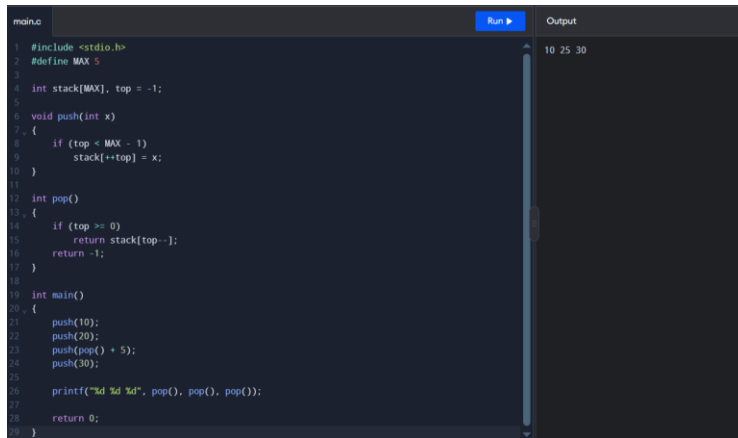
Question 1. ADT and function calls

main.c	Run ▶	Output
<pre>1 #include <stdio.h> 2 int update(int x) 3 { 4 x = x + 5; 5 return x; 6 } 7 int main() 8 { 9 int a = 10; 10 printf("%d ", update(a)); 11 printf("%d", a); 12 return 0; 13 }</pre>		15 10

Question 2. Structure-based ADT

main.c	Run ▶	Output
<pre>1 #include <stdio.h> 2 3 typedef struct { 4 int data; 5 int count; 6 } Item; 7 8 int main() 9 { 10 Item x = {5, 2}; 11 x.data += x.count++; 12 printf("%d %d", x.data, x.count); 13 return 0; 14 }</pre>		7 3

Question 3. Stack ADT using an array



```
main.cpp Run Output
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void push(int x)
7 {
8     if (top < MAX - 1)
9         stack[++top] = x;
10 }
11
12 int pop()
13 {
14     if (top >= 0)
15         return stack[top--];
16     return -1;
17 }
18
19 int main()
20 {
21     push(10);
22     push(20);
23     push(pop() + 5);
24     push(30);
25
26     printf("%d %d %d", pop(), pop(), pop());
27
28     return 0;
29 }
```

10 25 30

Question 4. Queue ADT

main.c	Run ▶	Output
<pre>1 #include <stdio.h> 2 #define MAX 5 3 4 int q[MAX], front = 0, rear = -1; 5 6 void enqueue(int x) 7 { 8 if (rear < MAX - 1) 9 q[++rear] = x; 10 } 11 12 int dequeue() 13 { 14 if (front <= rear) 15 return q[front++]; 16 return -1; 17 } 18 19 int main() 20 { 21 enqueue(4); 22 enqueue(8); 23 dequeue(); 24 enqueue(12); 25 26 printf("%d %d %d", dequeue(), dequeue(), dequeue()); 27 return 0; 28 }</pre>		<pre>-1 12 8</pre>

Question 5. ADT operation sequence

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int a[4] = {2, 4, 6, 8}; 6 int n = 4, i; 7 8 for (i = 1; i < n; i++) 9 a[i] = a[i] + a[i - 1]; 10 11 for (i = 0; i < n; i++) 12 printf("%d ", a[i]); 13 14 return 0; 15 }</pre>		2 6 12 20